5 程序实现

5.1 核心算法实现

5.1.1 中点画线法实现

```
/**
 * 中点画线法 (0 < k < 1的情况)
 * @param x0, y0 起点坐标
 * @param x1, y1 终点坐标
 * @return 直线上的像素点集合
 */
std::vector<D2D1_POINT_2F> MidpointLine(int x0, int y0, int x1, int
    y1) {
    std::vector<D2D1_POINT_2F> pixels;
    int dx = x1 - x0;  // x方向增量
    int dy = y1 - y0;  // y方向增量

    // 决策参数初始化: d = 2dy - dx (整数化后)
    int d = 2 * dy - dx;
    int incrE = 2 * dy;           // 选择E点时的增量
    int incrNE = 2 * (dy - dx);   // 选择NE点时的增量

    int x = x0, y = y0;
    pixels.push_back(D2D1::Point2F((float)x, (float)y));

    while (x < x1) {
        if (d < 0) {
            // d < 0: 中点在直线下方, 选择E点(x+1, y)
            d += incrE;
            x++;
        } else {
            // d >= 0: 中点在直线上方, 选择NE点(x+1, y+1)
            d += incrNE;
            x++;
            y++;
        }
        pixels.push_back(D2D1::Point2F((float)x, (float)y));
    }
    return pixels;
```